Pacgate AI - 双 AIPC 部署手册

> 在每台机器上克隆仓库，运行相同的安装步骤，两台机器即可完全运行 deer-flow 研究与 qm 协作。 版本 0.1.28 - 2026-10-13 英文版本：AIPC-DEPLOYMENT-HANDBOOK.md 前置条件：Docker Desktop、Ollama、Node.js 24+。 `install.ps1` 会拉取 `ollama-models.txt` 中列出的模型。

⚠️ 重要发现（2026-09-02）— 部署 AIPC #2 前请先阅读

以下问题是在 AIPC #1 试点期间发现的，**已在本仓库中修复。** AIPC #2 必须拉取**更新后**的代码（见 Stage 1），以获得这些修复。

> **2026-09-23 更新（取代 2026-09-15 的「两者均可克隆」提示）。** 两个仓库已**不再一致** —— `pacgate-ai/pacgate-ai-pr` 落后 26 个提交，缺少 workflow 接线修复（ `b7fc540` 、 `039afdc` ），克隆 fork 会得到 **10 个内置工作流，而不是公司的 222 个**，且不会报错。 **请克隆** `JZKK720/pacgate-ai-pr` （见 Stage 1）。向哪个仓库推送发布标签，仍决定 镜像进入哪个 GHCR 命名空间——见 `plans/012-master-release-namespace.md` 。

1. **deer-flow 代理无法查询 pacgate 的法律数据库。** 根本原因：没有工具接入 pacgate-api 的 `/api/kb/search` （RAG）或 `/api/search` （法律连接器），且 记忆适配器静默回退到本地文件（ `PacgateMemoryStorage` 需要 `PACGATE_MATTER_ID` ）。 **修复：** 新增 `pacgate-mcp` 服务（FastMCP），向 deer-flow 暴露 `pacgate_kb_search` 、 `pacgate_connector_search` 、 `pacgate_list_connectors` 。 已在 `deer-flow-extensions-config.json` 中与 openviking 并列注册。

2. **openviking MCP 内置了错误的 API 密钥。** `deer-flow-extensions-config.json` 使用了应用密钥（ `OPENVIKING_API_KEY` ），但 openviking 的 `root_api_key` 是 `OPENVIKING_ROOT_API_KEY` 。错误的密钥导致 openviking 返回 401，进而回滚了 **整个** MCP 工具加载（deer-flow 使用 `asyncio.gather` ），因此**没有**任何 MCP 工具出现。**修复：** 使用 `OPENVIKING_ROOT_API_KEY` （模板现已使用 `${OPENVIKING_ROOT_API_KEY}` ）。

3. `docker compose up -d --force-recreate deer-flow` **会清空 deer-flow 的本地数据库。** SQLite 数据库、管理员用户、线程和 `.jwt_secret` 位于 `/app/backend/.deer-flow/` **容器内部**（未挂载）。重建容器会丢失它们 → 前端返回 401 并出现 `/setup` 页面。 **配置变更请使用** `docker compose restart deer-flow` ；只有在你接受丢失本地 数据库时才重建（然后重新运行 `/setup` ）。

4. **QM 登录需要邮件传输，而不是 Outlook SMTP。** 旧的 SMTP 路径 （ `smtp.office365.com` + 应用密码）已失效——微软已于 2025 年 9 月停用 Exchange Online 的基本身份验证 / 应用密码。 `qm check` 失败并返回 `535 5.7.139 Authentication unsuccessful` 。**修复：** qm 的 auth 代理现在使用 **Resend** 传输（ `AUTH_EMAIL_TRANSPORT=resend` ）。你必须在 `deploy/qm-pacgate/.env` 中提供

`RESEND_API_KEY`（见 Stage 4）。此条最初称 Resend 是唯一选项，Stage 4 的双传输设置已取代——试点用 Mailpit，无需密钥。

5. **qm web-ui 无法自行认证。** 其服务器（`/app/server/index.ts`）设置 `AUTH_MODE = COOKIE_AUTH ? "dev" : "portal"`。因为设置了 `CORE_SIGNING_SECRET`， 它处于 **portal** 模式，需要 portal 签发的身份令牌。**没有**无需密钥的方式 直接访问 web-ui——你必须运行 `portal` + `auth`（Resend）或外部 OIDC 提供方。 `ADMIN_GRANTS` 是授权种子，不是登录方式。

6. **`pacgate-ai` 账号向 `JZKK720/pacgate-ai-pr` 推送被阻止**（403，需要 2FA 授权）。 **变通方案：** `pacgate-ai` 账号可以创建 fork 并推送到那里。fork `pacgate-ai/pacgate-ai-pr` 现在在 `main` 上携带所有修复。

架构：两台相同的机器

两台 AIPC 都运行完整的栈：

```
每台 AIPC 机器：
  nginx :8089  → pacgate-api :8080（Rust 元数据 API）
                → deer-flow  :8001（研究工作空间）
  Postgres :5432（本地元数据数据库）
  OpenViking :1933（长期记忆通道，MCP）
  qm :8182（协作工作空间，通过 `qm up` 运行）
  Ollama :11434（本地运行，GPU/NPU）
```

每台机器都是自包含且独立可运行的。任一机器上的律师都可以使用研究模式 （deer-flow，位于 `http://localhost:8089/research/`）和协作模式 （qm，位于 `http://localhost:8182`），无需依赖另一台机器。

如果之后希望在两台机器之间共享事项数据，请用私有网格（Tailscale 或 WireGuard） 连接它们，并决定同步或单一权威模型。这是试点后的决策，不是部署前置条件。

开始前需要准备什么

- 源码仓库访问权限 —— `JZKK720/pacgate-ai-pr` 或 `pacgate-ai/pacgate-ai-pr` 均可。 两者**均为公开**，单纯克隆无需任何认证；仅当需要推送时才需 PAT 或 `gh auth login`。
- 两台 AIPC 上都运行 Docker Desktop
- 两台 AIPC 上都运行 Ollama（`install.ps1` 会拉取它需要的模型）
- 如果使用带 cloud 标签的 deepseek 模型，每台 AIPC 上完成 `ollama signin`
- 两台 AIPC 上都安装 Node.js 24+（供 qm 使用）
- PowerShell 7（`pwsh`）可选 —— 安装脚本在系统自带的 PowerShell 5.1 上同样可以 运行。（在 2026-10-11 之前这一说法并不成立，且有两处问题：其一，只要附加了重定向， PS 5.1 就会把*任何*写入 stderr 的原生命令变成终止性错误，而 `docker compose up -d` 每次运行都会把进度写入

stderr，导致安装脚本在引导租户和管理员之前就中断；其二， 管理员注册调用使用了 PS 7 专有的 `-SkipHttpErrorCheck` 参数，5.1 会在请求发出之前 就拒绝它，于是每一次 API 调用都静默失败。现在 `install.ps1` 、 `setup-qm.ps1` 和 `smoke-deer-flow-outputs.ps1` 中所有写 stderr 的调用都经由 `Invoke-WithNativeStderr` ，所有 API 调用都经由 `Invoke-ApiRequest` 。详见 `deploy/DEFECT-ps51-native-stderr.md` 。）

- **无需** `docker login ghcr.io` ——Pacgate 运行时镜像以**公开** GHCR 包发布 （见 Stage 0）。

Stage 0：运行时镜像（开发机，已完成）

运行时已发布到 GHCR，AIPC 上无需重建。

当前版本（应拉取的版本）：

镜像	状态
`ghcr.io/jzkk720/pacgate-api:0.1.30`	已发布，公开。新增案件工作区汇总视图（ `GET /api/matters/:id/workspace` ）。
`ghcr.io/jzkk720/pacgate-mcp:0.1.30`	已发布，公开。向 deer-flow 暴露 19 个 MCP 工具（新增 `pacgate_get_workspace` 、 `pacgate_read_memory` 、 `pacgate_write_memory` ）。
`ghcr.io/jzkk720/deer-flow-pacgate:0.1.30`	已发布，公开。
`ghcr.io/jzkk720/deer-flow-frontend-pacgate:0.1.30`	已发布，公开。
`ghcr.io/jzkk720/ocr-service:0.1.30`	已发布，公开。PaddleOCR 抽取服务；自 0.1.16 起为一等镜像。自 0.1.28 起，其 PP-OCRv4 权重已**内置于镜像**，抽取无需外部模型站点，可离线运行。
`ghcr.io/volcengine/openviking@sha256:46f9e34c…`	在 `compose.prod.yaml` 中按摘要固定。上游公开镜像。

> 命名空间与版本于 2026-09-22 更正。此表此前列出 `ghcr.io/pacgate-ai/*` 的 0.1.0/0.1.3。 `pacgate-ai` 为遗留镜像，实际命名空间为 `jzkk720` ，发布自 plan 016 起已迁移。旧表中的 `pacgate-ai/ … -frontend-pacgate:0.1.0` 行标注"已发布"，实际返回 **404**——从未存在。五个 `jzkk720/*` 镜像按当前固定值（0.1.28，见上表）匿名拉取均返回 HTTP 200。

历史发布表（保留以追溯，已被取代）：

镜像	状态
`ghcr.io/pacgate-ai/pacgate-api:0.1.3`	当时已发布。修复 0.1.1 的容器网络 bug。
`ghcr.io/pacgate-ai/pacgate-mcp:0.1.3`	当时已发布。
`ghcr.io/pacgate-ai/deer-flow-pacgate:0.1.3`	当时已发布。
`ghcr.io/pacgate-ai/deer-flow-frontend-pacgate:0.1.0`	**从未发布——返回 404。** 请勿使用。

所有 Pacgate 包必须在 GHCR 上设置为公开可见，以便 AIPC 无需注册表凭据即可拉取。 上线前验证：

```
# 期望无需 docker login 即返回 HTTP 200。401/403 表示包仍是私有的。
# 404 表示该命名空间下不存在此标签——通常是 compose.prod.yaml 的镜像固定值
# 与实际发布位置不一致。
#
# （必须带 Accept 头——省略时公开清单会返回 404，而不是 200。）
#
# 本片段直接从 compose.prod.yaml 读取四个镜像固定值，因此永远不会过期：
# 此前该片段硬编码了一个已被移除的旧标签，把健康系统误报为故障。
$acc = "application/vnd.oci.image.index.v1+json,application/vnd.docker.distribution.manifest.
$compose = "deploy/client-bundle/compose.prod.yaml"
$pins = Select-String -Path $compose -Pattern "image:\s*(ghcr\.io/[^\s]+)" -AllMatches |
        ForEach-Object { $_.Matches } | ForEach-Object { $_.Groups[1].Value } |
        Where-Object { $_ -notmatch "openviking" } | Sort-Object -Unique

foreach ($pin in $pins) {
  $repo = $pin -replace "^ghcr\.io/", ""            # owner/name:tag
  $name = ($repo -split ":")[0]                       # owner/name
  $tag  = ($repo -split ":")[1]
  $t = (Invoke-RestMethod "https://ghcr.io/token?scope=repository:$name`:pull").token
  $code = (Invoke-WebRequest -Uri "https://ghcr.io/v2/$name/manifests/$tag" `
            -Headers @{Authorization="Bearer $t"; Accept=$acc} `
            -Method Head -UseBasicParsing).StatusCode
  "{0,-58} ⇒ HTTP {1}" -f $pin, $code
}
```

切换可见性（GitHub Web UI——个人账号的 API 路由返回 404）： GitHub → 你的个人资料 → Packages → `pacgate-api` → Package settings → Visibility → **Public** → Save。对 `deer-flow-pacgate` 重复此操作。这是安全的：镜像只包含 编译后的二进制文件和 SQL 迁移，所有密钥都在运行时通过 `.env` 注入，且安装程序 已拥有对相同代码的完整源码访问权限。

仅当 Rust 源码变更时，才在开发机上重建并推送。**标签值应从 compose 固定值中读取， 而不要手写版本号**——此处硬编码的标签已两次过期，其中一次还把健康系统误报为故障：

```
cd c:\Users\cubecloud-io\github-pr\pacgate-ai-pr
# 模式已改为当前命名空间（jzkk720）。旧写法匹配 'pacgate-ai/pacgate-api'，
# 在 compose.prod.yaml 中已无任何匹配行，$tag 会为空，导致下面的构建/推送
# 静默使用空标签。
$tag = (Select-String -Path deploy/client-bundle/compose.prod.yaml `
        -Pattern 'jzkk720/pacgate-api:(\S+)').Matches.Groups[1].Value
if (-not $tag) { throw 'could not read the image tag from compose.prod.yaml' }
docker build -t ghcr.io/jzkk720/pacgate-api:$tag -f pacgate-ai/Dockerfile ./pacgate-ai
docker push  ghcr.io/jzkk720/pacgate-api:$tag
```

实践中建议使用 `build-ghcr.yml` 工作流，以保证四个镜像版本一致——参见 `deploy/README-BUILD.md` 与 `plans/012-master-release-namespace.md`。

不要在 AIPC 上重建——试点运行已发布的摘要。

> **端口说明：** 栈将 nginx 绑定到主机端口 `8089`（`deploy/client-bundle/compose.prod.yaml` 中提交的值）。如果该端口在机器上已被占用，请编辑 `deploy/client-bundle/compose.prod.yaml` 中 `nginx` 的 `ports:` 条目，并在下方所有验证 URL 中使用新端口。

Stage 1：在每台 AIPC 上克隆仓库

在两台机器上：

```
cd C:\
git clone https://github.com/JZKK720/pacgate-ai-pr.git
cd pacgate-ai-pr
git remote -v   # origin 必须是 JZKK720/pacgate-ai-pr
```

> **AIPC #2 说明（2026-09-23 更正）：** 两个仓库**已不再一致**，因此「克隆哪一个 都可以」不再成立。**请克隆 `JZKK720`。**
>
> `pacgate-ai/pacgate-ai-pr` 落后 **26 个提交**（2026-09-23 核实），缺少 `b7fc540` 与 `039afdc`，因此仍带有**最初的 workflow 接线缺陷**。fork 中虽有那 15 个 workflow YAML，但未接入 `pacgate-api`，于是 API 只会提供**10 个内置工作 流，而不是公司的 222 个**，而且不会报任何错。上面「内容完全一致」的说法在写入 时是准确的，分歧发生在之后——这正是文档不应在没有可失败检查的情况下断言两者 同步的原因。
>
> 命名空间方面的差异没有变化：向哪个仓库推送标签，决定镜像发布到哪个 GHCR 命名空间。见

仓库为公开，克隆无需凭据；仅当需要推送时才使用个人访问令牌或 GitHub CLI（`gh auth login`）。

Stage 2：部署核心栈（两台机器，步骤相同）

在每台 AIPC 上运行这些步骤。Docker Compose 栈会启动 pacgate-api、Postgres、nginx 和 deer-flow。

```
cd C:\pacgate-ai-pr\deploy\client-bundle
copy .env.example .env
notepad .env
```

填写这些值：

```
PACGATE_DB_PASSWORD=<生成一个强密码>
PACGATE_JWT_SECRET=<生成一个随机十六进制字符串>
PACGATE_TENANT_ID=pacgate-law
OPENVIKING_ROOT_API_KEY=<生成一个 32 字符十六进制字符串>
OPENVIKING_API_KEY=<生成一个 32 字符十六进制字符串>
```

`OPENVIKING_API_KEY` 是**必需的**——安装程序会用它渲染 `deer-flow-extensions-config.json`，如果缺失或保留为 `change-me` 则报错停止。

如果需要，生成密钥：

```
# 数据库密码（16 位十六进制）
-join ((1..16) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })

# JWT 密钥（32 位十六进制）
-join ((1..32) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })

# OpenViking 密钥（每个 32 位十六进制）——每行生成一个新值
-join ((1..32) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })
```

运行安装程序：

```
.\install.ps1
```

安装程序会拉取（公开、无需登录的）GHCR 镜像，从 `.env` 密钥渲染 `OPENVIKING_CONF_CONTENT` 和 `deer-flow-extensions-config.json`，启动 Docker Compose 栈，并拉取 `ollama-models.txt` 中列出的 Ollama 模型。如果模型已拉取，此步骤很快。

验证核心栈：

```
docker compose -f compose.prod.yaml ps
curl http://localhost:8089/version
curl http://localhost:8089/pacgate/health
```

预期：八个容器全部运行（pacgate-db、pacgate-api、deer-flow、deer-flow-frontend、 pacgate-mcp、ocr-service、openviking、nginx），`/version` 返回 `{"version":"0.1.28","revision":"<git`

`sha>"}`，且 `/pacgate/health` 返回 `ok`。

> **不要在 nginx 根路径探测 `/health`。** nginx 按设计将 `/` 路由到 deer-flow 前端，`curl http://localhost:8089/health` 会返回前端的 404 页面——看似失败，实为正确路由。`/version` 也在根路径（映射到 API 的 `/build-info`）；只有 `/pacgate/*` 路径会到达 API。

Stage 3：初始化租户并开通账号（两台机器）

> **`install.ps1` 现已自动完成（step 6a）。** 在新版安装上无需手工执行，本节保留 用于恢复场景，其中的命令与安装程序实际运行的命令一致。

自 0.1.22 起的账号机制（开通人员前请先阅读）

- `POST /api/auth/register` 仅限**首位用户**：在全新部署上只创建一个账号 （引导管理员），此后一律返回 403。
- 此后的每个账号由管理员通过 `POST /api/auth/users` 创建（Bearer = 管理员令牌）。
- 检索工作区登录 = 邮箱 + 密码；协作（qm）登录 = 一次性邮件链接，受 `AUTH_ALLOWED_EMAILS` 名单控制。

先初始化租户，再由安装程序引导创建管理员：

```
# 初始化租户（幂等，slug 必须与 PACGATE_TENANT_ID 匹配）
docker exec pacgate-db psql -U pacgate -d pacgate -c "INSERT INTO tenants (name, slug) SELECT
```

开通律师账号（管理员账号存在后执行）：

```
$login = Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/login" -Method Post `
  -Body '{"email":"admin@pacgate-law.com","password":"<admin-password>"}' `
  -ContentType "application/json"
$body = @{email="<attorney-email>"; password="<attorney-password>"; role="attorney"} | Conver
Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/users" -Method Post `
  -Headers @{Authorization="Bearer $($login.token)"} -Body $body -ContentType "application/js
```

注册 qm 桥接服务账号（安装程序自动完成，此处保留手工方式用于恢复），使用相同的管理员开通路由：`email="qm-bridge@pacgate-law.com"`。

协作工作区侧：在 `deploy/client-bundle/qm-pacgate/.env` 中设置 `AUTH_ALLOWED_EMAILS`（逗号分隔）与 `ADMIN_GRANTS=<email>:org_admin`，然后重新运行 `setup-qm.ps1`。

Stage 3.5：验证 OpenViking 记忆服务（两台机器）

OpenViking 是长期记忆通道：deer-flow 和 qm 在那里存储对话上下文，并在后续会话中召回。 它作为 compose 栈的一部分启动。

```
curl http://localhost:1933/health
```

预期：`{"status":"ok","healthy":true,...}`。安装程序将 OpenViking 配置 （Ollama 嵌入 + VLM）渲染到 `.env` 中的 `OPENVIKING_CONF_CONTENT`，并在首次启动时 初始化服务器的 `ov.conf`。

功能检查（可选，使用 `.env` 中的根密钥）：

```
$key = (Get-Content .env | Select-String '^OPENVIKING_ROOT_API_KEY=').Line.Split('=')[1]
$body = '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
curl.exe -s -X POST http://localhost:1933/mcp -H "X-API-Key: $key" -H "Content-Type: applicat
```

预期：工具列表包含 `find`、`search`、`read`、`remember`。

边界规则：OpenViking 只存储对话上下文（决策、偏好、工作知识）。事项文档和 T1-T4 受控内容保留在 pacgate-api/pacgate-rag 中。

Stage 4：引导 qm（两台机器，步骤相同）

qm 独立于 Docker Compose 栈运行。在核心栈健康后，在每台机器上引导它。

```
cd C:\pacgate-ai-pr\deploy\client-bundle
.\setup-qm.ps1
```

脚本会提示输入： - 管理员工作邮箱（小写） - Pacgate 桥接邮箱：`qm-bridge@pacgate.local` - Pacgate 桥接密码：你在 Stage 3 中注册的那个

脚本会生成签名密钥，在 qm-pacgate 目录中创建 `.env`，用 `qm check` 验证配置， 用 `qm sandbox build` 构建沙箱镜像，取得（SHA-256 校验后的）核心容器所需的 静态 docker CLI，并构建 `qm-pacgate-sandbox-local` exec-daemon 包装镜像。

QM 登录需要邮件传输。 qm 的 auth 代理发送一次性登录链接。支持两种传输：

- **本地/试点拓扑（Mailpit SMTP 捕获器）：** `SMTP_HOST=mailpit SMTP_PORT=1025 AUTH_EMAIL_TRANSPORT=smtp SMTP_TLS=none`。链接落在 **Mailpit 收件箱**中——试点时 打开 `http://localhost:8025` 领取登录链接即可，不发送真实邮件。
- **生产拓扑（Resend）：** `AUTH_EMAIL_TRANSPORT=resend` 并在 `deploy/qm-pacgate/.env` 中设置 `RESEND_API_KEY`，`AUTH_EMAIL_FROM` 使用 Resend 已验证的发件人（Outlook 地址不属于已验证；微软已停用 Exchange Online 的基本身份验证/应用密码，因此原生 Outlook SMTP 不受支持）：

```
RESEND_API_KEY=re_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
AUTH_EMAIL_FROM="PacGate <onboarding@resend.dev>"
```

要么在 Resend 中验证一个真实域名（例如 `pacgate-law.com`），要么暂时使用 Resend 的测试发件人：

```
AUTH_EMAIL_FROM="PacGate <onboarding@resend.dev>"
```

启动 qm：

```
cd C:\pacgate-ai-pr\deploy\client-bundle\qm-pacgate
docker compose -f compose.qm.yaml up -d
```

> **请用 compose，不要用 `qm up`。** `@yc-software/qm` CLI 的 docker 生命周期 只有 POSIX 一条路——它的 `which()` 直接执行 `/bin/sh`，在 Windows 上不存在 （2026-10-06 实测：`execFileSync("/bin/sh", …)` → `ENOENT`），所以 `qm up`、`qm down`、`qm status` 在任何 AIPC 上都会报 "docker not found on PATH"。 `compose.qm.yaml` 描述的是同一套拓扑（同名容器、同名卷、同一网络），并且 额外承载 Pacgate 补丁： `patch/pi-models.ts` 与 `patch/local-sandbox.ts` bind-mount 覆盖核心源码、静态 docker CLI 与宿主 docker socket 均已挂载、 沙箱包装镜像已接线。这些挂载在每次 `down` / `up -d` 循环中都保持不变—— 模型路由与沙箱链路在 compose 路径上是持久的。不要对同一目录同时运行 `docker compose -f compose.qm.yaml up -d` 和 `qm up`（两者会争抢同名卷与网络）。

验证 qm：

```
# 打开 http://localhost:8182  (web-ui) - 需要 portal 身份令牌
# 打开 http://localhost:8181  (portal) - 登录前门
# 使用管理员邮箱登录（魔法链接经配置的邮件传输；试点为 Mailpit）
# 发送一条测试消息
# 询问："List available pacgate workflows"
```

> **Web-ui 认证现实：** qm web-ui （`:8182`）无法自行认证任何人——它必须通过 portal （`:8181`）访问，由 portal 签发身份令牌。如果你直接打开 `:8182`， 会看到 "reached through the portal."。始终通过 `:8181` 访问。
>
> **开发模式替代方案（试点 / 单用户）：** 对于本地试点，你可以让 qm 以 **dev/cookie 模式**运行，而不是 portal。用 `NODE_ENV=development` + `ALLOW_UNAUTHENTICATED_CORE=1` 且**不设置** `CORE_SIGNING_SECRET` 重建 `qm-pacgate-core`，并用**不设置** `CORE_SIGNING_SECRET` 重建 `qm-pacgate-web-ui`。 然后 `POST /signin` 直接在 `:8182` 用 `{"user":"<principal>"}` 工作，且无需 Resend 密钥。这**不是**生产正确配置（无认证）——仅用于单用户试点。

Stage 5：验证 deer-flow（两台机器）

在每台机器上，验证研究工作空间：

```
# 打开 http://localhost:8089/research/
# 选择或创建一个事项
# 询问："Summarize recent force majeure case law in China"
# 验证：回复包含引用
# 验证：回复已保存到事项记忆
```

Stage 5.5：全链路操作 — deer-flow ↔ QM ↔ OpenViking

两个工作空间共享**一个 OpenViking** 长期记忆通道和**一个 pacgate-api** 元数据存储。 本节说明两个系统如何相互通信，以及如何端到端验证该链路。

拓扑（2026-09-04 已验证）

```
pacgate-ai-bundle_default  (Docker Compose 网络)
├── openviking :1933   ← 长期记忆（MCP：find/search/read/remember）
├── pacgate-api :8080  ← 元数据 API（事项/工作流/连接器）
├── pacgate-mcp :8000  ← FastMCP 桥接，暴露 pacgate KB/连接器搜索
├── deer-flow :8001    ← 研究工作空间（消费 openviking + pacgate-mcp）
└── nginx :8089        ← 入口（deer-flow 前端 + /pacgate/ API）

qm-pacgate  (qm up 网络)
├── qm-pacgate-core    ← 协作代理运行时
├── qm-pacgate-web-ui  ← 浏览器聊天 UI（:8182）
└── qm-pacgate-pg      ← qm Postgres

桥接：qm-pacgate-core 也加入了 pacgate-ai-bundle_default，因此它可以访问
openviking:1933、pacgate-api:8080 和 host.docker.internal:11434（Ollama）。
```

链路如何工作

1. **deer-flow → OpenViking**：`deer-flow-extensions-config.json` 将 `openviking` 注册为 `http://openviking:1933/mcp` 的 HTTP MCP 服务器，使用**根** API 密钥（`OPENVIKING_ROOT_API_KEY`）。研究运行在那里存储和召回对话上下文。
2. **deer-flow → pacgate**：`pacgate-mcp`（FastMCP）向 deer-flow 暴露 `pacgate_kb_search` / `pacgate_connector_search` / `pacgate_list_connectors`， 由 pacgate-api 的 RAG + 法律连接器支撑。
3. **QM → OpenViking**：QM 核心有 `OPENVIKING_URL=http://openviking:1933` 和 `OPENVIKING_API_KEY`（根密钥）。 `pacgate-qm` 沙箱工具（`ov-remember` / `ov-search` / `ov-read`）在代理沙箱内通过 `http://host.docker.internal:1933` 调用 OpenViking，并带 `OPENVIKING_ACCOUNT=pacgate-law` + `OPENVIKING_USER`。

4. **QM → pacgate**：`pacgate-qm` 沙箱工具登录 pacgate-api（`PACGATE_API_EMAIL` / `PACGATE_API_PASSWORD`）以发现工作流、将 QM 作用域绑定到事项、读写事项记忆， 并执行工作流。

验证全链路

```
# 1. OpenViking MCP 响应（使用 .env 中的根密钥）
$key = (Get-Content .env | Select-String '^OPENVIKING_ROOT_API_KEY=').Line.Split('=')[1]
$body = '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
curl.exe -s -X POST http://localhost:1933/mcp -H "X-API-Key: $key" -H "Content-Type: applicat
# 预期：工具 find/search/read/remember

# 2. QM 核心访问 openviking + pacgate-api + Ollama
docker exec qm-pacgate-core sh -c "curl -s -o /dev/null -w 'openviking:%{http_code}\n' http://

# 3. QM 登录工作（dev/cookie 模式）
$body = '{"user":"admin@pacgate-law.com"}'
curl.exe -s -X POST http://localhost:8182/signin -H "Content-Type: application/json" -d $body
# 预期：{"ok":true,"user":"admin@pacgate-law.com"}
```

QM ↔ bundle 网络加入（幂等）

QM 核心已加入 bundle 网络，以便按名称解析 `openviking` 和 `pacgate-api`。 如果 `qm up` / `qm down` 循环丢失了加入，请重新应用：

```
docker network connect pacgate-ai-bundle_default qm-pacgate-core
```

> **注意：** QM 有意**不是** bundle 中的 Docker Compose 服务。它由 `@yc-software/qm` CLI（`qm up` / `qm down`）管理，有自己的生命周期。 网络加入是唯一的耦合——请保持这样。不要将 QM 重写为 compose 服务； 那会与 qm CLI 冲突，并在每次 `qm up` 时重建容器。

Stage 6：冒烟测试清单（两台机器）

在每台 AIPC 上独立运行此清单。

核心栈

- [ ] `docker compose -f compose.prod.yaml ps` 显示 8 个服务运行（含 openviking、pacgate-mcp、ocr-service、deer-flow-frontend）
- [ ] `curl http://localhost:8089/version` 返回发布版本与 revision JSON； `curl http://localhost:8089/pacgate/health` 返回 `ok`

- [ ] `curl http://localhost:1933/health` 返回健康 JSON
- [ ] Postgres 有 `pacgate-law` 租户
- [ ] 管理员用户可以在 `http://localhost:8089/api/auth/login` 登录
- [ ] deer-flow 在 `http://localhost:8089/research/` 返回真实研究回复

qm 协作

- [ ] `docker compose -f compose.qm.yaml ps` 显示 qm 服务运行中
- [ ] `http://localhost:8182` 加载 qm Web UI
- [ ] 管理员可以登录
- [ ] qm 可以列出 Pacgate 工作流类别
- [ ] qm 可以通过桥接执行一个 Pacgate 工作流

Ollama

- [ ] `ollama list` 显示所需模型
- [ ] deer-flow 可以调用 Ollama 进行推理
- [ ] qm 可以调用 Ollama 进行推理

数据

- [ ] `./data/tenants/` 目录存在且可写
- [ ] `./openviking/` 目录存在并在重启后持久化
- [ ] 通过 API 上传文档正常
- [ ] deer-flow 研究运行后事项记忆持久化
- [ ] 跨会话召回：通过 OpenViking `remember` 存储的事实可在后续会话中通过 `search` 召回

部署后管理栈

启动和停止

```
# 启动核心栈
docker compose -f compose.prod.yaml up -d

# 停止核心栈
docker compose -f compose.prod.yaml down

# 启动 qm
cd C:\pacgate-ai-pr\deploy\client-bundle\qm-pacgate
docker compose -f compose.qm.yaml up -d

# 停止 qm（去掉 -v：保留卷与数据）
docker compose -f compose.qm.yaml down
```

> `qm up` / `qm down` 在 Windows 上无法运行（CLI 的 `which()` 直接调 `/bin/sh` → ENOENT）；compose 是 AIPC 上唯一可用的 qm 生命周期，并且它承载了保证模型路由与沙箱链路持久的补丁。

更新到新版本

```
cd C:\pacgate-ai-pr\deploy\client-bundle
.\install.ps1 -Update
```

`-Update` 现在会自动完成以下全部工作，因此**不再需要单独执行 `git pull`**（那一步最容易被遗忘，而一旦遗忘，就会让新镜像配上旧配置静默运行）：

步骤	作用
1	刷新仓库工作区（仅快进 fast-forward）
2	从模板重新渲染 MCP 配置，变更前先备份
3	拉取新的 GHCR 镜像
4	重启技术栈，并重载 nginx

当机器上存在本地改动时，它会拒绝执行而不是猜测：

- 仓库有未提交改动 → 跳过仓库更新，并列出被改动的文件。**不会** stash、 reset 或丢弃任何内容。
- 存在远端没有的本地提交 → 不合并、不变基。仓库保持原样，其余更新继续进行。

如需仅更新镜像，可使用 `-SkipRepoPull`：

```
.\install.ps1 -Update -SkipRepoPull
```

qm 只做报告，不自动更新

如果本机运行了 qm， `-Update` 还会检查其沙箱镜像是否仍与源码一致：

```
[OK] qm sandbox matches its source (38e062ec7c5c ...)
```

或

```
[WARN] qm sandbox source has CHANGED since the image was pinned.
       qm is running OLD skills and tools. Rebuild + repin:
         cd deploy\qm-pacgate
         npm exec qm -- sandbox build   # 然后把打印出的 digest 重新固定
         pwsh -File ..\..\scripts\qm-sandbox-fingerprint.ps1 -Write
```

这一点很重要：qm 的智能体运行在一个**按 digest 固定**的沙箱镜像里（记录在 `deploy/qm-pacgate/qm.config.jsonc` ）。按 digest 固定本身是正确的 — 隔离边界 就应当不可变 — 但这意味着仓库更新可以改动 `deploy/qm-pacgate/sandbox/` ，而镜像完全不变。于是智能体继续使用旧的 skills 和 tools 运行，**任何地方都不会报错。**

`-Update` 对此**只报告、不自动重建**，因为重建需要 Node 24 + npm + buildx， 且之后必须重新固定 digest — 这属于配置变更，不应在客户机器上无人值守地执行。 一次错误的自动重建，比一条可见的警告更糟。

更新会保留数据： - `./data/tenants/` （卷挂载）— 事项、文档、记忆 - Postgres 数据（命名卷）— 元数据数据库

这台机器是不是最新版本？直接问它

```
curl.exe -s http://localhost:8089/version
```

```
{"version":"<release>","revision":"<git sha>"}
```

这里返回的是**编译进正在运行的 pacgate-api 二进制文件**的版本，以及它所基于的 commit。它故意不回显 compose 中的 pin 或镜像 tag：那些记录的是"曾部署了什么"， 而真正要捕获的故障，恰恰是所部署的产物与实际运行的进程不一致。

在此之前，无法区分一台最新机器和一台落后的机器 — 从外部看两者完全相同， 唯一的办法是 SSH 进去读 compose 文件，并寄希望于容器与之相符。

切换模型

deer-flow（研究工作空间）： 1. 编辑 `deer-flow-config.yaml` — 重新排序 `models` 列表（第一项 = 默认） 2. 重启： `docker compose -f compose.prod.yaml restart deer-flow`

qm（协作工作空间）： 1. 若模型集变更，编辑 `deploy/client-bundle/qm-pacgate/qm.config.jsonc` 2. 重建核心（compose 保留补丁挂载，路由持久）： `powershell cd C:\pacgate-ai-pr\deploy\client-bundle\qm-pacgate docker compose -f compose.qm.yaml up -d --force-recreate core`

注册新用户

`/api/auth/register` **仅限首位用户**，引导账号创建后一律返回 403（见 Stage 3）。 此后的每个账号通过管理员路由创建：

```
# /pacgate 前缀必填——见 Stage 3 的说明。Bearer = 管理员令牌
# (自 POST /pacgate/api/auth/login 获得，见 Stage 3 登录步骤)
$body = @{email="<user>@pacgate-law.com"; password="<password>"} | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/users" -Method Post `
  -Headers @{Authorization="Bearer <admin-token>"} -Body $body -ContentType "application/json
```

备份数据库

```
docker exec pacgate-db pg_dump -U pacgate pacgate > backup.sql
```

查看日志

```
docker compose -f compose.prod.yaml logs -f pacgate-api
docker compose -f compose.prod.yaml logs -f deer-flow
```

已知限制

- **`qm up` / `qm down` 在 Windows 上无法运行。** qm CLI 的 docker 生命周期 只有 POSIX 一条路：它的 `which()` 执行 `execFileSync("/bin/sh", …)`，在每台 Windows AIPC 上都是 `ENOENT`（2026-10-06 实测）。请使用上面的 compose 路径 （`docker compose -f compose.qm.yaml up -d`）——同样的拓扑与容器名，并且 承载 Pacgate 补丁，重建时不会丢失任何东西。
- 每台机器都有自己的独立 Postgres 和 `./data/tenants/` 目录。除非你之后添加 私有网格和同步或单一权威模型，否则事项数据不会在机器之间共享。
- PkuLaw 连接器令牌已过期。在 `https://mcp.pkulaw.com` 重新生成，并在试点期间 需要中国法律搜索时在 `.env` 中设置 `PKULAW_API_KEY`。
- 四个 WASM crate（citation-check、clause-parser、doc-validator、rule-engine） 仍是桩。这些是未来蓝图工作，不影响 Phase 1 试点功能。
- **模型选择：** API 默认使用目标机器上可能不存在的模型。Stage 3 之后，应用 按租户的模型覆盖，使 LLM 层级指向该机器 `ollama list` 中实际存在的模型。 推荐的试点集（2026-08-28 基准测

试）：`gemma4:12b-it-qat`（Main — 13 秒/工具轮， 模式有效的工具调用，端到端验证）、`qwen3.8:27b-mtp-q4_K_M`（Mid — 73 秒/工具轮， 批量表格审查质量更强）、`nomic-embed-text:latest`（嵌入）。交互层级避免使用 推理模式模型（例如 nemotron）——它们可能挂起长 docx 生成。SQL 模板见 `plans/007-aipc-full-installation-handoff.md` 附录 A。

引用的文件

文件	用途
`deploy/client-bundle/compose.prod.yaml`	pacgate-api + deer-flow + Postgres + nginx 的 Docker Compose
`deploy/client-bundle/install.ps1`	核心栈的一键 Windows 安装程序
`deploy/client-bundle/setup-qm.ps1`	qm 引导脚本（密钥、配置、沙箱构建）
`deploy/client-bundle/.env.example`	客户端密钥模板
`deploy/client-bundle/ollama-models.txt`	要预拉取的模型
`deploy/client-bundle/deer-flow-config.yaml`	deer-flow 多模型配置（5 个模型，可切换）
`deploy/qm-pacgate/qm.config.jsonc`	qm 本地部署配置
`deploy/SETUP-AND-OPERATIONS.md`	完整的 3 天现场安装指南（参考）
`deploy/DEPLOYMENT-GUIDE.md`	工程师级部署细节（参考）